



MTM4692

Spring 2026

Fettah KIRAN

Department of AI and Data

Engineering

fkiran@yildiz.edu.tr

ACID Transactions & Triggers

Atomicity, Consistency, Isolation, Durability — Automated Actions



Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:05	Recap Quiz	Review
1:05 – 1:30	ACID Properties	Theory 
1:30 – 1:55	Transaction Control	Theory 
1:55 – 2:15	Isolation Levels	Theory 
2:15 – 2:30	 Break (15 min)	
2:30 – 2:50	Triggers: Concepts	Theory 
2:50 – 3:10	BEFORE & AFTER Triggers	Theory 
3:10 – 3:25	SQLite Transactions & Triggers	Practice 
3:25 – 3:40	Practice & Homework	Practice 
3:40 – 3:45	Recap & next week preview	Wrap-up

Learning Objectives

By the end of this week you will be able to:

1. Explain the **ACID** properties of transactions
2. Use **COMMIT, ROLLBACK, SAVEPOINT** for transaction control
3. Understand **isolation levels** and concurrency problems
4. Create **BEFORE** and **AFTER** triggers
5. Use **NEW** and **OLD** row references in triggers
6. Implement **SQLite triggers** and transactions

Part 1: ACID Properties

What is a Transaction?

A **transaction** is a sequence of SQL operations treated as a **single logical unit**.

```
-- Transfer $100 from Alice to Bob
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE name = 'Alice';
UPDATE accounts SET balance = balance + 100 WHERE name = 'Bob';
COMMIT; -- Both succeed, or neither does
```

Either **both** operations succeed, or **neither** does.

ACID Properties

Property	Meaning
Atomicity	All or nothing — entire transaction succeeds or fails
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions don't interfere
Durability	Committed data survives crashes

These four properties guarantee **reliable** database transactions.

Atomicity — All or Nothing

```
START TRANSACTION;  
UPDATE accounts SET balance = balance - 100 WHERE name = 'Alice';  
-- ✗ System crash here!  
UPDATE accounts SET balance = balance + 100 WHERE name = 'Bob';  
COMMIT;
```

Without atomicity: Alice loses \$100 but Bob doesn't receive it!

With atomicity: The entire transaction is rolled back. Alice keeps her \$100.

Think of it as an "undo button" for the entire group of operations.

Consistency — Valid State to Valid State

```
-- Constraint: balance >= 0
START TRANSACTION;
UPDATE accounts SET balance = balance - 1000
WHERE name = 'Alice'; -- Alice only has $500
-- This would violate the constraint!
COMMIT;
```

Consistency means:

- All constraints are satisfied before AND after
- CHECK constraints, FOREIGN KEY, UNIQUE, NOT NULL
- Business rules are maintained

Isolation — No Interference

Transaction A (Bank Transfer):

```
START TRANSACTION;
```

```
UPDATE Alice balance = 400;
```

```
UPDATE Bob balance = 600;
```

```
COMMIT;
```

Transaction B (Balance Check):

```
SELECT balance FROM Alice;
```

```
-- Should see 500 or 400?
```

Isolation controls what Transaction B can see while A is running.

Different **isolation levels** provide different answers!

Durability — Survives Crashes

```
START TRANSACTION;  
INSERT INTO orders VALUES (...);  
COMMIT;  --  Data is now permanent!  
--  Server crashes 1 second later  
-- Data is STILL there after restart
```

Once `COMMIT` succeeds, the data is **permanently stored**.

- Written to **disk** (not just memory)
- Survives power failures, crashes, restarts



Exam Focus — ACID

Property	One-Line Summary
Atomicity	All statements succeed or all fail
Consistency	Constraints always satisfied
Isolation	Transactions don't see each other's uncommitted data
Durability	Committed data is permanent

Common exam question: "Which ACID property ensures that partial failures don't corrupt data?" → **Atomicity**

Part 2: Transaction Control

Transaction Commands

Command	Action
START TRANSACTION / BEGIN	Start a new transaction
COMMIT	Save all changes permanently
ROLLBACK	Undo all changes since START
SAVEPOINT name	Create a checkpoint
ROLLBACK TO name	Undo to a savepoint
RELEASE SAVEPOINT name	Remove a savepoint

Basic Transaction Flow

```
START TRANSACTION;
```

```
INSERT INTO orders (customer_id, total) VALUES (1, 250.00);
```

```
INSERT INTO order_items (order_id, product_id, qty) VALUES (LAST_INSERT_ID(), 5, 2);
```

```
-- Everything OK? Save it!
```

```
COMMIT;
```

```
START TRANSACTION;
```

```
INSERT INTO orders (customer_id, total) VALUES (1, 250.00);
```

```
INSERT INTO order_items (order_id, product_id, qty) VALUES (999, 5, 2);
```

```
-- ❌ Foreign key error!
```

```
ROLLBACK; -- Undo everything
```

SAVEPOINT — Partial Rollback

```
START TRANSACTION;
```

```
INSERT INTO orders VALUES (100, 1, '2025-01-01', 500.00);  
SAVEPOINT sp1;
```

```
INSERT INTO order_items VALUES (1, 100, 5, 2, 50.00);  
SAVEPOINT sp2;
```

```
INSERT INTO order_items VALUES (2, 100, 999, 1, 100.00);  
-- ✗ Product 999 doesn't exist!
```

```
ROLLBACK TO sp2;      -- Undo only the last INSERT  
-- sp1 and the order are still intact
```

```
INSERT INTO order_items VALUES (2, 100, 10, 1, 100.00);  
-- ✓ Correct product
```

```
COMMIT;  -- Save order + both valid items
```

SAVEPOINT Visualization

START TRANSACTION

— INSERT order —  Kept

— SAVEPOINT sp1

— INSERT item 1 —  Kept

— SAVEPOINT sp2

— INSERT item 2 —  ROLLBACK TO sp2

— INSERT item 2 (fixed)—  Kept

— COMMIT —  All saved

MySQL Auto-Commit Mode

By default, MySQL auto-commits each statement:

```
-- Each statement is its own transaction
INSERT INTO student VALUES (...); -- Auto-committed!
INSERT INTO student VALUES (...); -- Auto-committed!
```

To use explicit transactions:

```
-- Option 1: START TRANSACTION
START TRANSACTION;
-- statements...
COMMIT;

-- Option 2: Disable auto-commit
SET autocommit = 0;
-- statements...
COMMIT;
SET autocommit = 1;
```


Transaction in a Stored Procedure

```
DELIMITER //
```

```
CREATE PROCEDURE transfer_funds(  
    IN p_from INT, IN p_to INT,  
    IN p_amount DECIMAL(10,2),  
    OUT p_status VARCHAR(50)  
)  
BEGIN  
    DECLARE v_balance DECIMAL(10,2);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SET p_status = 'FAILED: Transaction rolled back';  
    END;  
  
    START TRANSACTION;  
  
    SELECT balance INTO v_balance  
    FROM accounts WHERE account_id = p_from;  
  
    IF v_balance < p_amount THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Insufficient funds';  
    END IF;  
  
    UPDATE accounts SET balance = balance - p_amount  
    WHERE account_id = p_from;  
  
    UPDATE accounts SET balance = balance + p_amount  
    WHERE account_id = p_to;  
  
    COMMIT;  
    SET p_status = 'SUCCESS';  
END //
```

```
DELIMITER ;
```

Part 3: Isolation Levels

Concurrency Problems

When multiple transactions run simultaneously:

Problem	Description
Dirty Read	Reading uncommitted data from another transaction
Non-Repeatable Read	Same query returns different results within a transaction
Phantom Read	New rows appear between two queries in same transaction

Dirty Read Example

```
Transaction A:  
START TRANSACTION;  
UPDATE student SET gpa = 4.0  
WHERE student_id = 1;
```

```
ROLLBACK;  
-- gpa is back to 3.5
```

Transaction B:

```
-- Reads gpa = 4.0 (uncommitted!)  
SELECT gpa FROM student  
WHERE student_id = 1;
```

```
-- But B already used 4.0!
```

Transaction B read data that was **never committed** — a **dirty read**.

Non-Repeatable Read Example

```
Transaction A:  
START TRANSACTION;  
SELECT gpa FROM student  
WHERE student_id = 1;  
-- Returns 3.5  
  
SELECT gpa FROM student  
WHERE student_id = 1;  
-- Returns 3.8 – different!  
COMMIT;
```

```
Transaction B:  
  
UPDATE student SET gpa = 3.8  
WHERE student_id = 1;  
COMMIT;
```

Same query, **different results** within the same transaction.

Phantom Read Example

```
Transaction A:  
START TRANSACTION;  
SELECT COUNT(*) FROM student  
WHERE gpa > 3.5;  
-- Returns 5  
  
SELECT COUNT(*) FROM student  
WHERE gpa > 3.5;  
-- Returns 6 – a "phantom" row!  
COMMIT;
```

```
Transaction B:  
  
  
INSERT INTO student (gpa)  
VALUES (3.9);  
COMMIT;
```

New rows **appeared** between two identical queries.

Four Isolation Levels

Level	Dirty Read	Non-Repeatable	Phantom
READ UNCOMMITTED	✔ Possible	✔ Possible	✔ Possible
READ COMMITTED	✘ Prevented	✔ Possible	✔ Possible
REPEATABLE READ	✘ Prevented	✘ Prevented	✔ Possible
SERIALIZABLE	✘ Prevented	✘ Prevented	✘ Prevented

MySQL default: REPEATABLE READ

PostgreSQL default: READ COMMITTED

Setting Isolation Level

```
-- For the current session
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- For the next transaction only
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Check current level
SELECT @@transaction_isolation;
```

Trade-off: Higher isolation = more safety, but lower performance



Exam Focus — Isolation Levels

READ UNCOMMITTED — Can see uncommitted data (dirty reads)

READ COMMITTED — Only sees committed data

REPEATABLE READ — Same query gives same result (MySQL default)

SERIALIZABLE — Full isolation, like running one at a time

Exam Tip: "Which level prevents dirty reads but allows phantoms?" → **READ COMMITTED**

Part 4: Triggers — Concepts

What is a Trigger?

A **trigger** is a stored program that **automatically executes** when a specific event occurs on a table.

Event (INSERT/UPDATE/DELETE)



TRIGGER
executes

← Automatic!

Triggers are **never called directly** — they fire automatically.

Trigger Components

Every trigger has three parts:

Component	Options	Description
Timing	BEFORE / AFTER	When the trigger fires
Event	INSERT / UPDATE / DELETE	What triggers it
Table	Any table	Which table to monitor

6 possible combinations:

- BEFORE INSERT , AFTER INSERT ,
- BEFORE UPDATE , AFTER UPDATE ,
- BEFORE DELETE , AFTER DELETE

Trigger Syntax (MySQL)

```
DELIMITER //
```

```
CREATE TRIGGER trigger_name  
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
    ON table_name  
    FOR EACH ROW  
BEGIN  
    -- trigger body  
    -- can reference NEW and OLD  
END //
```

```
DELIMITER ;
```

FOR EACH ROW = the trigger fires **once per affected row**

NEW and OLD References

Event	NEW	OLD
INSERT	✓ New row values	✗ Not available
UPDATE	✓ New values	✓ Old values
DELETE	✗ Not available	✓ Deleted row

```
-- In an UPDATE trigger:  
NEW.gpa    -- the value AFTER the update  
OLD.gpa    -- the value BEFORE the update
```



Exam Focus — NEW and OLD

INSERT: Only `NEW` available (no old row exists)

UPDATE: Both `NEW` and `OLD` available

DELETE: Only `OLD` available (row is being removed)

Exam Question: "In a BEFORE UPDATE trigger, what does `OLD.gpa` refer to?"

Answer: The value of gpa **before** the update

Part 5: BEFORE & AFTER Triggers

BEFORE Triggers — Validation & Modification

BEFORE triggers execute **before** the DML operation.

Use cases:

- **Validate** data before insert/update
- **Modify** values before they're saved
- **Prevent** operations (SIGLAL error)

```
DELIMITER //
```

```
CREATE TRIGGER validate_gpa  
  BEFORE INSERT ON student  
  FOR EACH ROW  
BEGIN  
  IF NEW.gpa < 0 OR NEW.gpa > 4.0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'GPA must be between 0.0 and 4.0';  
  END IF;
```

BEFORE INSERT: Auto-Fill Fields

```
DELIMITER //
```



```
CREATE TRIGGER student_before_insert  
  BEFORE INSERT ON student  
  FOR EACH ROW  
BEGIN  
  -- Auto-set creation timestamp  
  SET NEW.created_at = NOW();  
  
  -- Capitalize name  
  SET NEW.first_name = CONCAT(  
    UPPER(LEFT(NEW.first_name, 1)),  
    LOWER(SUBSTRING(NEW.first_name, 2))  
  );  
  
  -- Default GPA if not provided  
  IF NEW.gpa IS NULL THEN  
    SET NEW.gpa = 0.0;  
  END IF;  
END //
```



```
DELIMITER ;
```


BEFORE UPDATE: Prevent Invalid Changes

```
DELIMITER //
```



```
CREATE TRIGGER prevent_gpa_decrease  
  BEFORE UPDATE ON student  
  FOR EACH ROW  
BEGIN  
  -- Prevent GPA from decreasing more than 1.0  
  IF OLD.gpa - NEW.gpa > 1.0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'GPA cannot decrease by more than 1.0';  
  END IF;  
  
  -- Auto-update timestamp  
  SET NEW.updated_at = NOW();  
END //
```



```
DELIMITER ;
```

AFTER Triggers — Logging & Side Effects

AFTER triggers execute **after** the DML operation.

Use cases:

- **Audit logging** (who changed what, when)
- **Updating related tables** (denormalization)
- **Notifications** (insert into message queue)

AFTER INSERT: Audit Log

```
DELIMITER //
```



```
CREATE TRIGGER student_after_insert  
  AFTER INSERT ON student  
  FOR EACH ROW  
BEGIN  
  INSERT INTO audit_log (  
    table_name, action, record_id,  
    new_values, action_time, action_user  
  ) VALUES (  
    'student', 'INSERT', NEW.student_id,  
    CONCAT('name=', NEW.first_name, ' ', NEW.last_name,  
          ', gpa=', NEW.gpa),  
    NOW(), CURRENT_USER()  
  );  
END //
```



```
DELIMITER ;
```

AFTER UPDATE: Track Changes

```
DELIMITER //
```



```
CREATE TRIGGER student_after_update  
  AFTER UPDATE ON student  
  FOR EACH ROW  
BEGIN  
  -- Only log if GPA actually changed  
  IF OLD.gpa != NEW.gpa THEN  
    INSERT INTO gpa_change_log (  
      student_id, old_gpa, new_gpa,  
      change_date, changed_by  
    ) VALUES (  
      NEW.student_id, OLD.gpa, NEW.gpa,  
      NOW(), CURRENT_USER()  
    );  
  END IF;  
END //
```



```
DELIMITER ;
```

AFTER DELETE: Cascade Cleanup

```
DELIMITER //
```



```
CREATE TRIGGER student_after_delete  
  AFTER DELETE ON student  
  FOR EACH ROW  
BEGIN  
  -- Log the deletion  
  INSERT INTO audit_log (table_name, action, record_id, old_values)  
  VALUES ('student', 'DELETE', OLD.student_id,  
          CONCAT('name=', OLD.first_name, ' ', OLD.last_name));  
  
  -- Update department student count  
  UPDATE department  
  SET student_count = student_count - 1  
  WHERE dept_id = OLD.dept_id;  
END //
```



```
DELIMITER ;
```

BEFORE vs AFTER Summary

Aspect	BEFORE	AFTER
Timing	Before DML executes	After DML executes
Can modify NEW?	✅ Yes	❌ No
Can prevent operation?	✅ Yes (SIGNAL)	❌ Already done
Audit logging	⚠️ Not ideal	✅ Perfect
Data validation	✅ Perfect	⚠️ Too late
Update related tables	⚠️ Risky	✅ Safe



Exam Focus — BEFORE vs AFTER

BEFORE INSERT → Validate + modify NEW values

AFTER INSERT → Log the insert, update counters

BEFORE UPDATE → Validate changes, set timestamps

AFTER UPDATE → Log changes (compare OLD vs NEW)

BEFORE DELETE → Validate if deletion is allowed

AFTER DELETE → Log deletion, cleanup related data

Managing Triggers

```
-- List triggers
SHOW TRIGGERS;

-- Triggers for a specific table
SHOW TRIGGERS LIKE 'student';

-- Drop a trigger
DROP TRIGGER IF EXISTS validate_gpa;

-- View trigger definition
SHOW CREATE TRIGGER student_after_insert;
```


Trigger Restrictions (MySQL)

1. **One trigger** per timing/event/table combination
 - Can't have TWO `BEFORE INSERT ON student`
2. Triggers **cannot call** stored procedures that return result sets
3. Triggers **cannot use** COMMIT/ROLLBACK
4. Triggers **cannot** create/drop tables
5. Triggers fire for **each row** affected

Part 6: SQLite Transactions & Triggers

SQLite Transaction Support

SQLite fully supports transactions:

```
BEGIN TRANSACTION;  
INSERT INTO student (first_name, last_name, dept_id, gpa)  
VALUES ('Test', 'Student', 1, 3.5);  
COMMIT;
```

```
-- Or ROLLBACK  
BEGIN TRANSACTION;  
DELETE FROM student WHERE gpa < 2.0;  
ROLLBACK; -- Undo!
```

SQLite SAVEPOINT

```
BEGIN TRANSACTION;

SAVEPOINT sp1;
INSERT INTO student VALUES (NULL, 'A', 'Test', 1, 3.0);

SAVEPOINT sp2;
INSERT INTO student VALUES (NULL, 'B', 'Test', 1, 3.5);

ROLLBACK TO sp2;  -- Only B is undone
COMMIT;           -- A is saved
```

Same concept as MySQL!

SQLite Isolation

SQLite uses a **simpler** model:

- **Single writer** at a time (no concurrent writes)
- Readers don't block writers, writers don't block readers (WAL mode)
- Effectively **SERIALIZABLE** isolation

```
-- Enable WAL mode for better concurrency  
PRAGMA journal_mode = WAL;
```

SQLite Triggers

SQLite supports triggers with slightly different syntax:

```
CREATE TRIGGER trigger_name
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
    ON table_name
    [FOR EACH ROW]      -- Optional (it's the default)
    [WHEN condition]    -- SQLite-specific!
BEGIN
    -- trigger body
END;
```

Key differences from MySQL:

- No `DELIMITER` needed
- Supports `WHEN` clause (conditional trigger)
- Uses `RAISE()` instead of `SIGNAL`

SQLite BEFORE INSERT Trigger

```
CREATE TRIGGER validate_student_gpa
  BEFORE INSERT ON student
BEGIN
  SELECT CASE
    WHEN NEW.gpa < 0.0 OR NEW.gpa > 4.0
    THEN RAISE(ABORT, 'GPA must be between 0.0 and 4.0')
  END;
END;
```

```
-- Test it
INSERT INTO student VALUES (NULL, 'Bad', 'GPA', 1, 5.0);
-- Error: GPA must be between 0.0 and 4.0
```

SQLite AFTER Trigger with Audit Log

```
CREATE TABLE audit_log (  
    log_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    action TEXT, table_name TEXT,  
    record_id INTEGER,  
    old_val TEXT, new_val TEXT,  
    timestamp TEXT DEFAULT (datetime('now'))  
);  
  
CREATE TRIGGER log_gpa_change  
    AFTER UPDATE OF gpa ON student  
    WHEN OLD.gpa != NEW.gpa  
BEGIN  
    INSERT INTO audit_log (action, table_name, record_id, old_val, new_val)  
    VALUES ('UPDATE', 'student', NEW.student_id,  
            'gpa=' || OLD.gpa, 'gpa=' || NEW.gpa);  
END;
```

Note: `WHEN OLD.gpa != NEW.gpa` — only fires when GPA actually changes!

SQLite RAISE() Function

Instead of MySQL's `SIGNAL SQLSTATE :`

```
-- RAISE options:
RAISE(IGNORE);           -- Skip this row
RAISE(ABORT, 'message');  -- Abort current statement
RAISE(ROLLBACK, 'message'); -- Rollback entire transaction
RAISE(FAIL, 'message');   -- Fail but keep prior changes

-- Example
CREATE TRIGGER prevent_delete_admin
    BEFORE DELETE ON users
    WHEN OLD.role = 'admin'
BEGIN
    SELECT RAISE(ABORT, 'Cannot delete admin users');
END;
```

SQLite Trigger: Auto-Increment Counter

```
-- Maintain a department student counter
CREATE TRIGGER inc_dept_count
  AFTER INSERT ON student
BEGIN
  UPDATE department
  SET student_count = COALESCE(student_count, 0) + 1
  WHERE dept_id = NEW.dept_id;
END;

CREATE TRIGGER dec_dept_count
  AFTER DELETE ON student
BEGIN
  UPDATE department
  SET student_count = COALESCE(student_count, 0) - 1
  WHERE dept_id = OLD.dept_id;
END;
```

MySQL vs SQLite Trigger Comparison

Feature	MySQL	SQLite
DELIMITER	Required	Not needed
FOR EACH ROW	Required	Optional (default)
WHEN clause	✗ Not supported	✓ Supported
Error raising	<code>SIGNAL SQLSTATE</code>	<code>RAISE(ABORT, msg)</code>
Multiple per event	✗ One per timing/event	✓ Multiple allowed
INSTEAD OF	✗ No	✓ Yes (for views)

Part 7: Practice Exercises

Exercise 1: Transaction Control

Write a MySQL transaction that:

1. Starts a transaction
2. Inserts a new course
3. Creates a SAVEPOINT
4. Inserts 3 enrollment records
5. Rolls back to the SAVEPOINT (undo enrollments)
6. Commits (keep the course, discard enrollments)

Exercise 2: BEFORE Trigger

MySQL (Theory):

Write a `BEFORE INSERT` trigger on `enrollment` that:

- Checks if the student's GPA is ≥ 2.0
- Checks if the course has available seats
- Raises an error if either condition fails

SQLite (Practice):

Write the equivalent trigger using `RAISE(ABORT, ...)`.

Exercise 3: AFTER Trigger for Audit

MySQL (Theory):

Write an `AFTER UPDATE` trigger on `student` that:

- Logs changes to an `audit_log` table
- Records: `student_id`, `old_gpa`, `new_gpa`, `change_date`, who changed it
- Only fires when GPA actually changes

SQLite (Practice):

Write the same trigger using SQLite's `WHEN` clause.

Exercise 4: Isolation Level Identification

For each scenario, identify the minimum isolation level needed:

#	Scenario	Level?
1	Must not see uncommitted data	?
2	Same SELECT must return same rows	?
3	No new rows can appear between queries	?
4	Maximum performance, dirty reads OK	?

Exercise 4: Answers

#	Scenario	Level
1	Must not see uncommitted data	READ COMMITTED
2	Same SELECT must return same rows	REPEATABLE READ
3	No new rows can appear	SERIALIZABLE
4	Maximum performance, dirty reads OK	READ UNCOMMITTED

Homework

1. MySQL Theory:

- Write a complete fund transfer procedure with ACID transactions
- Create triggers for audit logging on 3 tables
- Explain isolation level trade-offs in a short essay

2. SQLite Practice:

- Implement a transaction with SAVEPOINTS
- Create BEFORE and AFTER triggers for validation and logging
- Test RAISE with different options (ABORT, ROLLBACK, FAIL)



Key Concepts

Topic	What to Remember
ACID	Atomicity, Consistency, Isolation, Durability
Atomicity	All or nothing
SAVEPOINT	Partial rollback within a transaction
Isolation Levels	RU → RC → RR → S (increasing protection)
Dirty Read	Reading uncommitted data
BEFORE trigger	Validate/modify, can prevent operation
AFTER trigger	Log/audit, update related tables
NEW/OLD	INSERT → NEW, UPDATE → both, DELETE → OLD
SIGNAL	Raise custom error in MySQL

Week 14: Backup, Restore, Replication & High Availability

- Backup strategies (logical vs physical)
- mysqldump and binary logs
- Replication types (async, semi-sync, group)
- RPO and RTO concepts
- High availability architectures
- SQLite backup methods

Thank You!

Any Questions?
